

Knowledge Base to Training Samples

Deep dive with one concrete example | Generated: 2026-08-25

Scope

This document explains exactly how text knowledge is transformed into detector training samples in this codebase, using seed 20260821 round 1 -> round 2 as a worked example.

Worked Example Snapshot

- Run timestamp: 20260824T105805Z
- Round 1 family: account_takeover | recall=0.6625 | f1=0.7969924812030075
- Round 2 family: trusted_device | recall=0.6875 | f1=0.8088235294117647
- Round 2 family selection mode: agent_1_adaptive_recommendation

Step 1: Knowledge Retrieval

LocalKnowledgeBase reads project text sources and retrieves top excerpts by term overlap against a query composed from current intent + prior weakness context.

- Data files: data/knowledge_base/seed_fraud_typologies.txt and data/knowledge_base/sources/*.txt
- Retriever implementation: src/mastercard_defence/rag.py
- Output type: evidence excerpts (text), not numeric rows

Sample retrieved-style text

Public seed knowledge for the first local demonstration.

Payment fraud can combine social engineering, account takeover, trusted-device abuse, beneficiary manipulation, and low-and-slow transaction patterns. Defenses should compare amount, time, device, beneficiary, channel, and short-window velocity signals. This seed is a synthetic planning aid and is not

How Metrics Flow Into Agent 1

Metrics never enter Agent 1 as raw numbers used for math. They enter as text, embedded inside memory strings and inside a structured WeaknessReport object. The real round-1 to round-2 chain below shows this mechanism.

- Agent 3 analyze(detection, fidelity) builds a WeaknessReport; see src/mastercard_defence/agents.py
- AttackMemory.add(...) persists hypothesis/specification/evaluation/weakness as JSON text; see src/mastercard_defence/memory.py
- AttackMemory.recent_context() returns strings like 'evaluation: {...}' and 'weakness: {...}', embedding the actual metric values as text
- ClosedLoop.run builds the next research_query by appending the last 4 memory strings (which contain those metrics) to a fixed query; see src/mastercard_defence/loop.py
- recommend_family(weakness, candidates, memory) picks the next attack_family: HeuristicAgents matches keywords in the weakness text (e.g. 'device' -> trusted_device, 'velocity' -> low_and_slow); QwenAgents instead sends the WeaknessReport fields as JSON to the LLM and parses back a FamilyRecommendation
- The chosen family is force-applied: hypothesis.attack_family = chosen_family in loop.py, so metrics indirectly steer AttackHypothesis by constraining which family Agent 1 is allowed to describe

Real round-1 evaluation metrics (input to Agent 3)

```
{
  "detection": {
    "precision": 0.9622641509433962,
    "recall": 0.6375,
    "f1": 0.7669172932330827,
    "roc_auc": 0.9553750000000001,
    "false_positive_rate": 0.02,
    "by_attack_family": {
      "account_takeover": {
        "precision": 1.0,
        "recall": 0.6375,
        "f1": 0.7786,
        "support": 80
      }
    }
  },
  "fidelity": {
```

```

"amount_mean_delta": 36.1666,
"amount_std_delta": 8.6931,
"behavioural_signal_delta": 1.45,
"channel_coverage": 1.0,
"behavioural_plausibility": 0.4171
}
}

```

Real round-1 WeaknessReport produced by Agent 3 (stored in Attack Memory)

```

{
  "round_id": 1,
  "observed_weaknesses": [
    "Detector recall is weak on the current synthetic attack family."
  ],
  "supporting_evidence": [
    "F1=0.767",
    "fidelity=0.417"
  ],
  "priority": "high",
  "recommended_next_attack_direction": "Test trusted-device and low-and-slow variants with reduced velocity signals.",
  "confidence": 0.65
}

```

Step 2: Agent 1 Converts Evidence to AttackHypothesis

Agent 1 receives retrieved evidence + attack memory context (which embeds the previous WeaknessReport and evaluation) and outputs a structured AttackHypothesis. The attack_family field is fixed by the controller to the family chosen via recommend_family; Agent 1 fills in the remaining narrative fields.

Field-by-field meaning

- attack_id: unique identifier for this round's attack line, e.g. round-2-social_engineering
- attack_family: one approved taxonomy family, forced by the controller from the family recommendation
- scenario: one-sentence description of the payment situation being simulated
- behavioural_mechanism: why the attack is expected to evade the current detector
- novelty_rationale: why this direction differs from recent memory context
- research_direction: short label matching the chosen family's narrative direction
- evidence: the EvidenceReference list carried over from knowledge-base retrieval (unchanged, not authored by the LLM)
- memory_context: last 4 raw memory strings, i.e. text containing prior evaluation/weakness JSON

Real AttackHypothesis recorded for round 2 (source: artifacts/attack_memory_full_20260824T065500Z.sqlite)

```

{
  "attack_id": "round-2-social_engineering",
  "attack_family": "social_engineering",
  "scenario": "Synthetic social-engineering account recovery abuse payment pattern",
  "target_context": "Offline synthetic payment security stress test",
  "behavioural_mechanism": "Fraudulent activity is shaped to reduce reliance on one obvious signal.",
  "novelty_rationale": "Selects a direction not used in the prior memory context.",
  "research_direction": "social-engineering account recovery abuse",
  "evidence": [
    {
      "source_id": "pci_payment_security_overview",
      "title": "PCI Security Standards Council payment-security overview",
      "excerpt": "Payment lifecycle, channel taxonomy, secure-by-design and mitigation framing (defensive summary excerpt).",
    }
  ],
  "memory_context": [
    "weakness: {...round-1 WeaknessReport JSON...}",
    "evaluation: {...round-1 detection+fidelity JSON...}"
  ]
}

```

Step 3: Agent 2 Converts Hypothesis to AttackSpecification

Agent 2 receives only the Agent 1 hypothesis fields (not detector metrics) and produces generation constraints: temporal_pattern, amount_pattern, device_pattern, beneficiary_pattern, feature_constraints, realism_constraints, and evasion_objective.

- Schema: AttackSpecification in src/mastercard_defence/contracts.py

- Implementation: HeuristicAgents.specify / QwenAgents.specify in src/mastercard_defence/agents.py
- This stage is still symbolic/structured; no detector rows yet
- feature_constraints and realism_constraints are the bridge consumed later by the generator's family profile in src/mastercard_defence/synthetic.py

Real AttackSpecification recorded for round 2 (paired with the hypothesis above)

```
{
  "attack_id": "round-2-social_engineering",
  "attack_family": "social_engineering",
  "scenario": "Synthetic social-engineering account recovery abuse payment pattern",
  "target_context": "Offline synthetic payment security stress test",
  "temporal_pattern": "mixed hours with a short burst for validation",
  "amount_pattern": "moderate amounts with controlled variance",
  "device_pattern": "new device mix",
  "beneficiary_pattern": "new beneficiary mix",
  "feature_constraints": {
    "synthetic_only": true
  },
  "realism_constraints": [
    "stay within synthetic schema",
    "retain attack labels and round metadata"
  ],
  "evasion_objective": "Expose detector reliance on a single behavioural feature."
}
```

Step 4: Generator Produces Detector-Ready Rows

The synthetic generator converts specification to tabular rows with the exact detector feature columns and label.

- Implementation: src/mastercard_defence/synthetic.py
- For trusted_device family: device_change probability is pushed low (approximately 0.08)
- Output includes amount, hour, device_change, beneficiary_change, velocity_24h, channel, is_fraud

Detector Input Table (Example Rows)

Rows below are example detector inputs (legitimate + trusted_device attack rows) generated with project logic.

attack_family	amount	hour	device_change	beneficiary_change	velocity_24h	channel	is_fraud
legitimate	41.88	10	0	0	2	mobile	0
legitimate	21.74	16	0	0	1	web	0
legitimate	57.12	8	0	0	3	mobile	0
legitimate	34.60	20	0	0	1	card_present	0
trusted_device	109.12	19	0	1	5	mobile	1
trusted_device	93.77	21	0	1	4	web	1
trusted_device	137.56	18	0	0	6	mobile	1
trusted_device	101.24	22	0	1	5	card_present	1

Step 5: Detector Training and Evaluation

FraudDetector trains on legitimate reference + train-attack rows, then evaluates on legitimate holdout + unseen attack rows.

- Detector implementation: src/mastercard_defence/detector.py
- Model: OneHotEncoder(channel) + HistGradientBoostingClassifier(random_state=42)
- Thresholding: predicted fraud if probability >= 0.5

Metric formulas

- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1 = $2 * (Precision * Recall) / (Precision + Recall)$
- False Positive Rate = $FP / (FP + TN)$
- ROC-AUC = area under ROC curve over score thresholds

Fidelity + Diversity Evaluator

This evaluator checks generated attack realism-like behavior and variation separately from detector metrics.

- Implementation: `evaluate_fidelity` and `evaluate_diversity` in `src/mastercard_defence/synthetic.py`
- Fidelity uses mean/std amount deltas, behavior deltas, and channel coverage
- Diversity uses unique-row ratio, channel entropy, family coverage ratio

Fidelity formula used in code

```
plausibility = max(0, 1 - min(1, 0.25*(mean_delta/ref_mean) + 0.25*(std_delta/ref_std) + 0.25*behaviour_delta + 0.25*(1 - channel_coverage)))
```

Why Knowledge Is Indirect

In this architecture, text knowledge never becomes labels directly. It influences strategy (hypothesis/specification), and the generator materializes that strategy into synthetic rows used by the detector.

File Map To Reproduce

- `data/knowledge_base/manifest.json`
- `data/knowledge_base/seed_fraud_typologies.txt`
- `data/knowledge_base/sources/nist_genai_profile_2024.txt`
- `src/mastercard_defence/rag.py`
- `src/mastercard_defence/loop.py`
- `src/mastercard_defence/contracts.py`
- `src/mastercard_defence/synthetic.py`
- `src/mastercard_defence/detector.py`
- `adaptive/adaptive_qwen_ctgan_results_20260824T105805Z.json`

Round 1 record excerpt

```
{
  "seed": 20260821,
  "round": 1,
  "attack_family": "account_takeover",
  "recall": 0.6625,
  "f1": 0.7969924812030075,
  "family_decision": {
    "source": "seeded_plan",
    "family": "account_takeover",
    "reason": "Initial family selected from the reproducible seeded plan."
  }
}
```

Round 2 record excerpt

```
{
  "seed": 20260821,
  "round": 2,
  "attack_family": "trusted_device",
  "recall": 0.6875,
  "f1": 0.8088235294117647,
  "family_decision": {
    "source": "agent_1_adaptive_recommendation",
    "recommended_family": "trusted_device",
    "recommendation_type": "approved_family",
    "reason": "The detection model has high precision but low recall for account takeover attacks, indicating it misses many instances of this attack type.",
    "target_weakness": "Detection model has high precision but low recall for account takeover attacks, indicating it misses many instances of this attack type.",
    "confidence": 0.85
  }
}
```